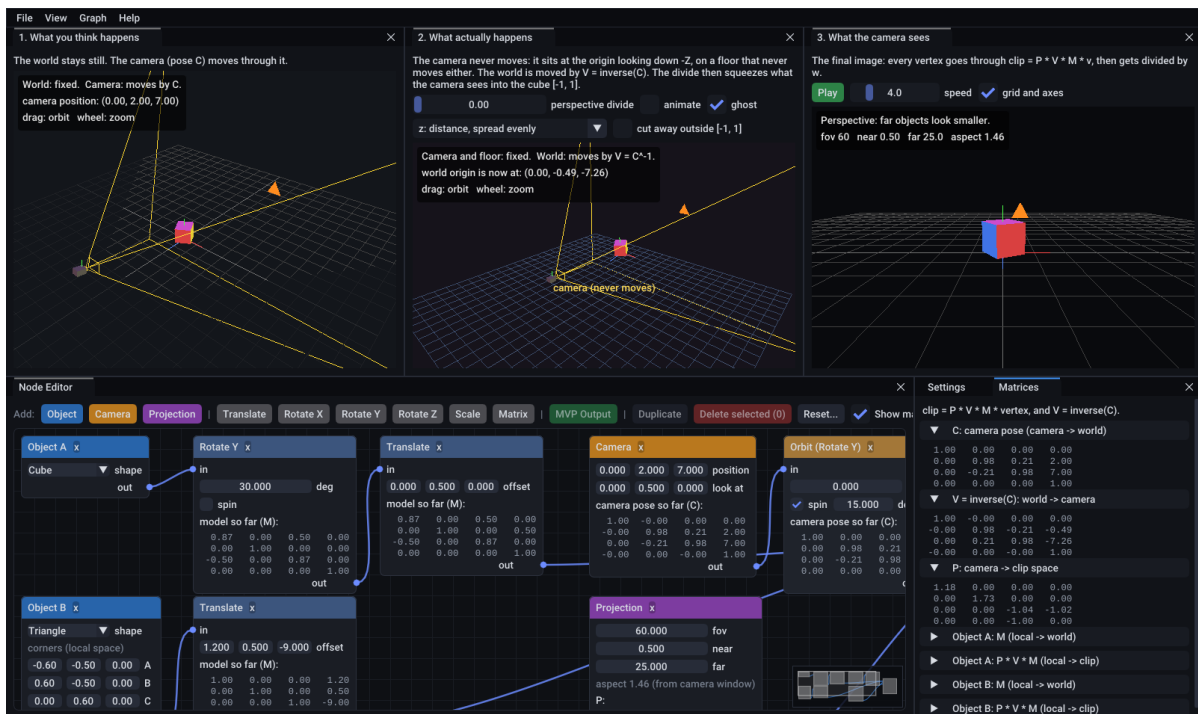


MVP Visualizer

How to use it, and how it works: the math, the shaders and the code

A guide to the C++ / OpenGL / Dear ImGui application in [src/](#). The first part shows how to use the program. The rest explains how a vertex travels from a mesh to a pixel, why the camera never really moves, how the three views fake and then reveal the perspective divide, and where each piece lives in the code.



The app with the starter example loaded. Top, left to right: what you think happens, what actually happens, what the camera sees. Bottom: the node editor and the Settings / Matrices panels.

- 1 The idea in one page
- 2 Using the program
- 3 The math of the pipeline
- 4 How the code is organised
- 5 The node graph
- 6 Rendering infrastructure
- 7 The shaders, line by line
- 8 The three views
- 9 The user interface
- 10 Extending the app

1. The idea in one page

Every 3D program asks the same question for each vertex of each mesh: *where on the screen does this point land?* The answer is a chain of matrix multiplications usually written as

$$clip = P \cdot V \cdot M \cdot vertex$$

read right to left. **M** (model) places the mesh in the world. **V** (view) re-expresses the world relative to the camera. **P** (projection) squeezes what the camera can see into a fixed box. The GPU then divides by the fourth coordinate w , which is what makes distant things small, and maps the box onto the window.

The trap for beginners is the word *camera*. When you move a camera in an engine, it feels like the camera moves through a still world. The GPU has no camera. There is only one fixed observer, at the origin, looking down the negative z axis. To make it look as if the camera moved right, the whole world is moved left. That is all the view matrix does: it is the **inverse** of the camera's placement in the world.

The app shows this with three windows driven by the same node graph:

Window	What it shows	Space it draws in
1. What you think happens	A still world and a camera model flying through it, placed by its pose C .	World space
2. What actually happens	The camera nailed to the origin and the world moved by $V = C^{-1}$. A slider then applies the perspective divide, turning the frustum into the cube $[-1, 1]^3$.	Camera (eye) space
3. What the camera sees	The real rendering: $P \cdot V \cdot M \cdot v$ straight to the window. You can also fly this camera.	Clip space, then pixels

The node graph builds the matrices. **Object** chains build each model matrix, a **Camera** chain builds the pose **C**, a **Projection** node builds **P**, and the **MVP Output** node collects them. A **Matrix** node holds any 4×4 matrix and can stand in for any of them. If no projection is connected, **P** is the identity, which shows what rendering looks like with only $V \cdot M$.

2. Using the program

2.1 Building and running

The build fetches GLFW, GLM, Dear ImGui (docking branch) and imnodes automatically. Only the OpenGL loader, GLAD, is a manual step: generate it at glad.dav1d.de (C/C++, OpenGL 3.3, Core profile, "Generate a loader") and copy it to `external/glad/`, as the README describes. Then:

```
cmake -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j
./build/gfx_node_editor
```

The first configure takes a while because it clones the dependencies. The build copies two fonts into `build/fonts/`. The window layout is saved to `imgui.ini` in the folder you start the program from, and restored next time.

2.2 The screen



The main window with the starter example. The numbers match the table below.

#	Part	What it is for
1	Menu bar	File (quit), View (show or hide windows, text toggles, reset layout), Graph (starter example or empty graph), Help (controls).
2	1. What you think happens	The world as you imagine it: a fixed scene with the camera and its viewing volume (yellow) placed in it. Drag to orbit, wheel to zoom.
3	2. What actually happens	The camera fixed at the origin and the world moved around it. Drag to orbit, wheel to zoom.
4	3. What the camera sees	The final rendered image. Has the Play button for flying the camera.
5	Play controls	Play / Stop, fly speed, and the <i>grid and axes</i> toggle for this view.

#	Part	What it is for
6	Divide controls	The perspective-divide slider, <i>animate</i> , <i>ghost</i> , the depth mode and <i>cut away outside</i> $[-1, 1]$.
7	Node editor toolbar	Buttons to add each node type, Duplicate, Delete selected, Reset, Show matrices, and (?) for help.
8	Node canvas	Where you build the graph. A minimap sits in the bottom-right corner. Warnings appear above the canvas.
9	Settings / Matrices	Settings holds every toggle. Matrices lists C, V, P and each object's M and $P \cdot V \cdot M$.

Every window can be moved. Drag a tab or title bar onto the docking targets to split or tab it, or drag it outside the main window to make it a separate OS window, for example on a second monitor. **View > Reset layout** restores this arrangement.

2.3 Your first scene, step by step

The program starts with an empty graph holding only the green **MVP Output** node. Its three input pins are Model, View and Projection. Build a scene like this:

1. Click **Object** in the toolbar. Drag from its *out* pin to the Output's **Model** pin. A cube appears at the world origin in all three views.
2. Click **Translate**. Connect Object → Translate → Model, replacing the direct link, and set the offset to (0, 0.5, 0). The cube now sits on the grid.
3. Click **Camera** and connect it to the **View** pin. Change *position* and *look at*. View 1 shows the camera moving through the world. In view 2 the camera stays put and the world moves the other way.
4. Click **Projection** and connect it to the **Projection** pin. View 3 gains perspective: things further away get smaller. Try the *fov*, *near* and *far* fields.
5. Add a **Rotate Y** between Camera and the View pin and tick *spin*. The camera orbits the cube in view 1, while in view 2 the world turns the opposite way around the fixed camera.
6. In view 2, drag the **perspective divide** slider from 0 to 1. The yellow frustum turns into the white cube $[-1, 1]^3$ and far objects shrink.
7. Press **Play** in view 3 and fly with WASD while dragging the image to look. The Camera node's fields change as you move.

The order of nodes matters. Object → Rotate → Translate rotates the object first and then moves it. Swap the two transforms and the object is moved first and then swung around the world origin. Section 3.1 explains why.

Graph > Load starter example builds a complete scene in one click: a rotated cube, a triangle further away, an orbiting camera and a projection. **Graph > New empty graph** starts over.

2.4 Editing the graph

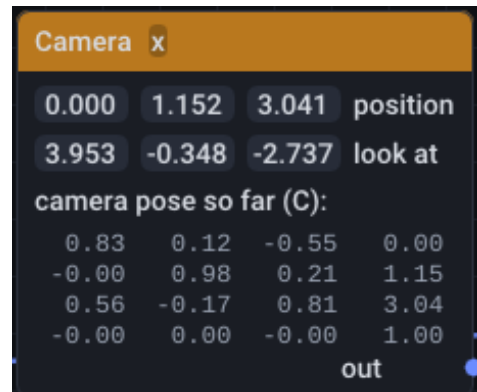
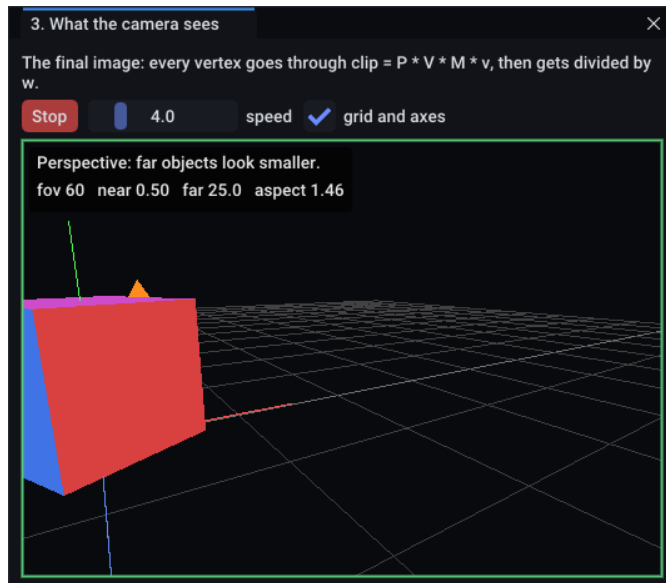
To...	Do this
Add a node	Click its toolbar button (it appears mid-canvas), or right-click empty canvas and pick it (it appears at the mouse).
Connect pins	Drag from an <i>out</i> pin to an input pin. An input takes one link, so a new link replaces the old one. The Output's Model pin takes many, one per object.
Remove a link	Ctrl+click it and drop it anywhere, or right-click it and choose Delete link, or select it and press Delete.
Remove a node	Click the x in its title bar, or right-click it and choose Delete, or select it and press Delete or Backspace.
Select several	Drag a box on empty canvas, then use the toolbar's Delete selected or Duplicate.
Duplicate	Toolbar Duplicate or right-click > Duplicate. Settings are copied, links are not.
Pan the canvas	Middle-mouse drag.
See matrices	<i>Show matrices</i> prints each node's running matrix: the model, pose or projection built so far.

2.5 The nodes

Node	Fields	Produces
Object	<i>shape</i> : Cube or Triangle. For a triangle, the three corners A, B, C in the object's own space, and <i>Reset triangle</i> .	Starts a model chain. Connect it, possibly through transforms, to Model.
Translate	<i>offset</i> (x, y, z)	<code>T(offset)</code>
Rotate X / Y / Z	<i>deg</i> ; <i>spin</i> animates the angle at <i>deg/s</i>	A rotation about that axis
Scale	<i>factor</i> (x, y, z)	<code>S(factor)</code>
Matrix	Any 4×4 matrix, typed row by row as on paper. <i>Identity</i> , <i>Transpose</i> , <i>Invert</i> (disabled when the determinant is 0), and the determinant shown below.	Exactly that matrix. Works anywhere: in an object chain, as or in the camera chain, as the projection, or after a Projection node.
Camera	<i>position</i> , <i>look at</i>	Starts a camera chain. Its pose is <code>C = inverse(lookAt(position, look at, up))</code> .
Projection	<i>fov</i> , <i>near</i> , <i>far</i> . Aspect is taken from the camera view's window.	A perspective matrix <i>P</i> .
MVP Output	Pins: Model (many), View, Projection	Collects M, C and P. Shows <code>V = inverse(C)</code> .

Node title colours tell you what a chain is: blue for a model chain, orange for a camera chain, purple for a projection chain, grey for a chain with no source, and red for an invalid chain (a cycle).

2.6 Flying the camera (play mode)



Play mode after flying forward and turning right. The green border shows play is on, and the Camera node's position and look-at fields have changed to match.

Press **Play** in *What the camera sees*. While playing, and while that window has focus:

Input	Effect
W / S	Fly forward / backward along the view direction
A / D	Strafe left / right
Q / E	Move down / up along the world's vertical
Shift	Three times faster
Drag the image	Look around (turn and tilt)
Esc or Stop	Leave play mode
speed slider	Base speed in units per second

Every move is written back into the node graph, so views 1 and 2, the Matrices panel and the node fields all follow. The View chain must start with a **Camera** or a **Matrix** node. Transforms after it are kept, and the start node is solved so that the whole chain lands on your pose. If nothing is connected to View, a Camera node is created and linked for you. Spinning nodes pause while you fly, so they don't fight your movement.

2.7 Keyboard and mouse reference

Where	Input	Effect
Anywhere	H	Show or hide the explanations on the views
Anywhere	L	Show or hide the labels drawn in the 3D scene
Views 1 and 2	Drag / mouse wheel	Orbit / zoom the observer camera
View 3, playing	WASD, Q/E, Shift, drag, Esc	Fly the camera (section 2.6)
Node editor	Delete or Backspace	Remove the selected nodes and links
Node editor	Ctrl+click a link	Detach it
Node editor	Right-click	Context menu for a node, a link or the empty canvas
Node editor	Middle drag	Pan

2.8 Settings

Setting	Effect
Play animations	Turns all <i>spin</i> animations on or off.
Dim what the camera can't see	In views 1 and 2, fades everything outside the student camera's volume.
Grid and axes in the camera view	Hides the floor grid and axes in view 3 so only the objects remain.
Explanations / Labels	The same switches as H and L.
Perspective divide	The view 2 slider, its animation, the ghost of the undivided scene, <i>cut away outside</i> [-1, 1], and the depth mode.
Show matrices on nodes	The running matrix printed on each node.

2.9 Experiments worth trying

- **Order of operations.** Swap a Rotate and a Translate in an object chain and watch the object orbit the origin instead of spinning in place.
- **The inverse.** Put a Translate (5, 0, 0) after the Camera. In view 2 the world origin moves to x = -5: the overlay prints it.
- **No projection.** Delete the Projection link. Only a 2×2×2 box around the camera is visible, nothing shrinks, and depth runs backwards (section 3.5).
- **Depth precision.** Set *near* to 0.01 and choose *z: real NDC z* in view 2. Almost everything is squeezed against the far face.
- **A custom projection.** Feed a Matrix into the Projection pin, for example the orthographic matrix `glm::ortho(-6, 6, -4, 4, 1, 15)` shown in section 9.1: rows (0.17, 0, 0, 0), (0, 0.25, 0, 0), (0, 0, -0.14, -1.14), (0, 0, 0, 1). The yellow volume becomes a box and distance no longer changes size.
- **Flip the image.** Put a Matrix after the Projection node with -1 in the top-left corner. View 3 is mirrored left to right.
- **Fly into the frustum.** Use play mode to fly between objects and watch the frustum move in view 1 while view 2's world slides past the fixed camera.

2.10 Warnings and problems

Message or symptom	Meaning and fix
Nothing is connected to Model	No object reaches the Output. Connect an Object chain (or any chain) to the Model pin.
Nothing is connected to View (using a default camera)	A default camera at (0, 2, 7) is used. Connect a Camera node to control it.
... is part of a cycle	Links form a loop, shown as red nodes. Delete one of the links.
View pin: the camera pose has determinant 0	The pose can't be inverted into V. Usually a Scale with a 0 factor, or an all-zero Matrix.
Projection: P has determinant 0	It still renders, but its volume can't be drawn in views 1 and 2.
Play stops with a message	The View chain must start with a Camera or a Matrix node.
Windows dragged outside appear in odd places	On Linux the app uses X11 (XWayland under Wayland) for this. A pure Wayland session can't place separate windows.
Fonts look plain	The fonts weren't found next to the executable. Rebuild, or run the program from the build folder.

3. The math of the pipeline

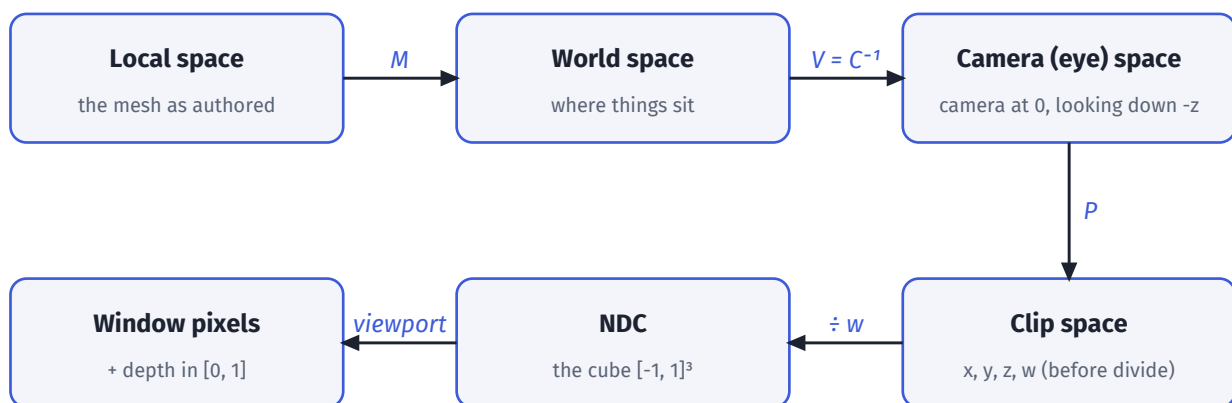


Figure 1. The coordinate spaces a vertex passes through, and the step between each pair.

All points are 4-component *homogeneous* vectors $(x, y, z, 1)$. The fourth component lets a 4×4 matrix express translation, and later carries the depth used for the perspective divide. GLM, like GLSL, stores matrices column by column, so `m[3]` is the fourth column: the translation part.

3.1 The model matrix M

Each transform node produces one 4×4 matrix in `Node::RecomputeLocal` (NodeGraph.cpp). With $s = \sin \theta$ and $c = \cos \theta$:

$$T(t) = \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad R_y(\theta) = \begin{bmatrix} c & 0 & s & 0 \\ 0 & 1 & 0 & 0 \\ -s & 0 & c & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad S(k) = \begin{bmatrix} k_x & 0 & 0 & 0 \\ 0 & k_y & 0 & 0 \\ 0 & 0 & k_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

`glm::translate`, `glm::rotate` and `glm::scale` build exactly these. Rotate X and Rotate Z are the same idea around the other axes. A Matrix node simply returns the matrix you typed.

Composition order

Matrices apply right to left, so $T \cdot R$ rotates first and translates second. The graph reads left to right in *data-flow order*: `Object` → `Rotate` → `Translate` means *first rotate, then translate*. To make that come out right, each node multiplies its own matrix on the **left** of everything upstream:

$$cumulative_{node} = local_{node} \cdot cumulative_{upstream}$$

so the chain above yields $M = T \cdot R$. That line is in `NodeGraph::Resolve`. Swap the two nodes and you get $R \cdot T$: the cube is moved first and then swung around the world origin.

3.2 The camera pose C and the view matrix V

The Camera node stores a *position* e and a *look-at target*. `glm::lookAt` builds the view matrix from three orthonormal axes:

- forward $f = \text{normalize}(\text{target} - e)$
- right $r = \text{normalize}(f \times \text{up})$, with $\text{up} = (0, 1, 0)$
- true up $u = r \times f$

$$V = \text{lookAt}(e, \text{target}, \text{up}) = \begin{bmatrix} r_x & r_y & r_z & -r \cdot e \\ u_x & u_y & u_z & -u \cdot e \\ -f_x & -f_y & -f_z & f \cdot e \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The rows are the camera's axes and the last column subtracts the camera position. The camera's **pose**, its placement in the world, is the inverse. Its columns are the axes and the position, which is easy to read:

$$C = V^{-1} = \begin{bmatrix} r_x & u_x & -f_x & e_x \\ r_y & u_y & -f_y & e_y \\ r_z & u_z & -f_z & e_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The app thinks in terms of C , because that is what a person means by "where the camera is".

`Node::CameraBasePose` returns `inverse(lookAt(...))`, guarding two degenerate cases: a target equal to the position, and looking straight up or down, where $f \times \text{up}$ is zero. Transforms after the Camera node then move the camera exactly as they move an object. `NodeGraph::Evaluate` finally sets $V = \text{inverse}(C)$.

Why the world moves the other way

Inverting a product reverses and inverts each factor: $(R \cdot C)^{-1} = C^{-1} \cdot R^{-1}$. So if a Rotate Y of +30° orbits the camera, the world in view 2 turns by -30°. If the camera moves +5 along x, **V** contains a translation of -5. View 2's overlay prints **V**'s translation column as "world origin is now at", which is exactly where the world origin ends up relative to the fixed camera.

3.3 The projection matrix P

The Projection node calls `glm::perspective(fov, aspect, n, f)`, the standard OpenGL right-handed projection with depth mapped to [-1, 1]. With $g = 1 / \tan(\text{fov} / 2)$ and a the aspect ratio:

$$P = \begin{bmatrix} g/a & 0 & 0 & 0 \\ 0 & g & 0 & 0 \\ 0 & 0 & (f+n)/(n-f) & 2fn/(n-f) \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

The last row is the important one. It copies -z into w. Since the camera looks down -z, the distance in front of the camera is $d = -z$, so after projection $w = d$. The GPU divides x, y and z by w, the *perspective divide*, to get normalised device coordinates (NDC):

$$x_{ndc} = (g/a) \cdot x / d \quad y_{ndc} = g \cdot y / d \quad z_{ndc} = (f+n)/(f-n) - 2fn/((f-n) \cdot d)$$

The $1/d$ in x and y is perspective: an object twice as far away covers half as much of the screen. With the default 60° field of view, $g \approx 1.732$. A 1-unit tall object at $d = 2$ spans 0.87 of NDC height, and at $d = 8$ only 0.22, four times smaller.

The depth formula also contains $1/d$, so depth is *not* linear. With the default near = 0.5 and far = 25:

distance d	0.5 (near)	1	2	5	12.75 (halfway)	25 (far)
z_{ndc}	-1.00	0.02	0.53	0.84	0.96	1.00

Half of the depth range is spent on the first half unit in front of the near plane. This is why a tiny near value ruins depth precision, and why view 2 offers two depth modes (section 7).

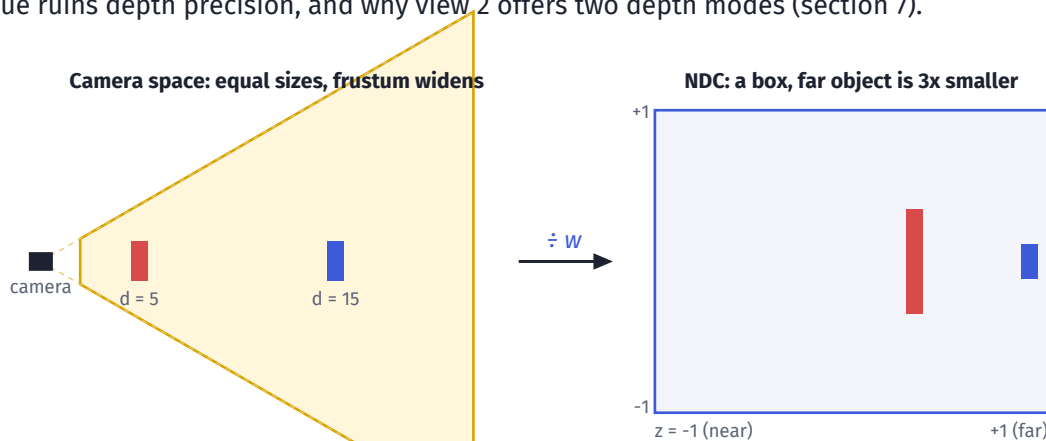


Figure 2. Two equal objects in camera space (left) and after the divide (right). The frustum becomes a box, the far object shrinks by $1/d$, and depth crowds towards the far face.

3.4 Clipping, depth and the window

The GPU keeps a point only if it lies inside the clip volume, tested *before* the divide:

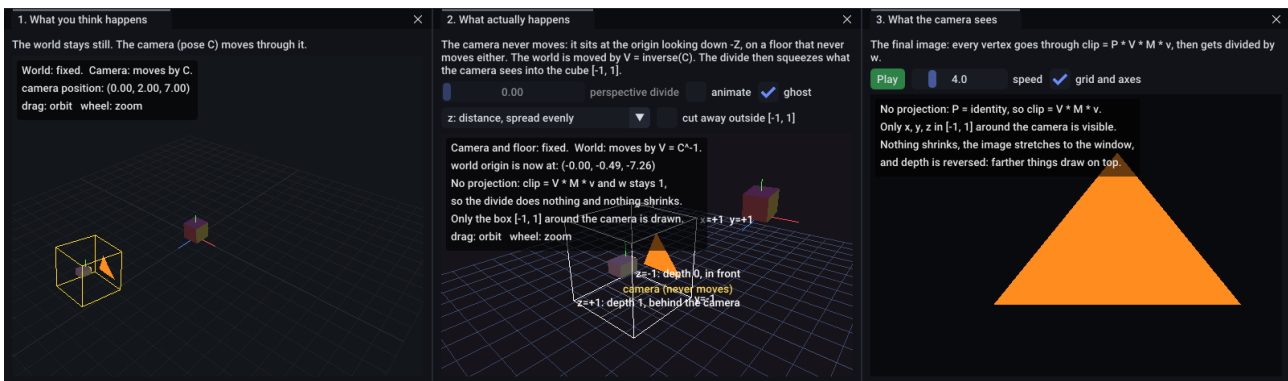
$$-w \leq x \leq w, -w \leq y \leq w, -w \leq z \leq w, \text{ with } w > 0$$

which is the same as requiring the NDC point to lie in the cube $[-1, 1]^3$. Survivors are mapped to the window: x and y from $[-1, 1]$ to pixels, and z to a depth value $(z_{\text{ndc}} + 1) / 2$ in $[0, 1]$. The depth test keeps the fragment with the smallest depth.

3.5 No projection at all

If nothing is connected to the Projection pin, Evaluate sets P to the identity, so $\text{clip} = V \cdot M \cdot \text{vertex}$ with $w = 1$. Then:

- **Nothing shrinks.** Dividing by $w = 1$ changes nothing, so distance has no effect on size.
- **Only a $2 \times 2 \times 2$ box is visible.** The clip test becomes $-1 \leq x, y, z \leq 1$ in camera space itself: one unit around the camera in every direction.
- **The image stretches.** No $1 / a$ factor corrects for the window's aspect ratio.
- **Depth is reversed.** Depth is $(z + 1) / 2$. A point one unit *in front* ($z = -1$) gets depth 0, the nearest value, and a point one unit *behind* gets depth 1. Further-away objects therefore draw on top of nearer ones, and things just behind the camera are drawn too.

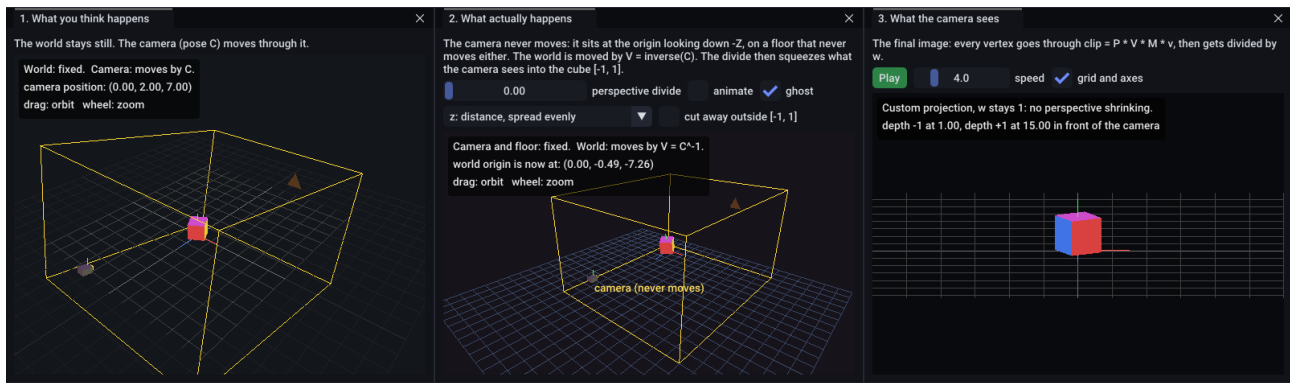


With no Projection connected: the camera's visible volume is the small box around it (views 1 and 2), and only the triangle that sits inside it appears in view 3, stretched to the window.

3.6 Any matrix as the projection

The Projection pin accepts any chain, so P can be a Matrix node, or a Projection node followed by more transforms ($\text{Projection} \rightarrow \text{Matrix}$ gives $P' = \text{Matrix} \cdot P$). The views can't rely on fov, near and far any more, so everything they need is read back out of P itself, using its inverse:

- **The visible volume** is whatever P maps onto the NDC cube, so its eight corners are $P^{-1} \cdot (\pm 1, \pm 1, \pm 1)$, each divided by its own w . That gives a pyramid for a perspective matrix, a box for an orthographic one, and the $2 \times 2 \times 2$ box for the identity.
- **Near and far** are the distances in front of the camera of the centres of the NDC faces $z = -1$ and $z = +1$, mapped the same way. For `glm::perspective` this returns exactly the node's values.
- **Perspective or not:** P divides by distance only if its bottom row reads z into w . The app checks the bottom row and flags the result as `perspective`.



A Matrix node holding an orthographic matrix as P: the viewing volume is a box, and an object's size no longer depends on its distance.

4. How the code is organised

File	Responsibility
<code>main.cpp</code>	Creates the window, the OpenGL context, ImGui, imnodes; the menu bar; the docking space; the frame loop.
<code>NodeGraph.h/.cpp</code>	Pure data and math: nodes, links, chain evaluation into an <code>EvalResult</code> , animation, writing a camera pose back, the starter example.
<code>Renderer.h/.cpp</code>	Everything OpenGL: the shader program, meshes and shape builders, offscreen framebuffers, the <code>Renderer</code> , the shared <code>Scene</code> , the orbit camera.
<code>UI.h/.cpp</code>	Everything ImGui that is not a 3D view: options, theme and fonts, small widgets, window titles and default layout, Node Editor, Settings, Matrices.
<code>Views.h/.cpp</code>	The three 3D windows. Each is a struct owning its framebuffer and, for views 1 and 2, its orbit camera. View 3 also owns play mode.

Only `NodeGraph` knows nothing about rendering, and only `Renderer` talks to OpenGL directly (apart from one `glPolygonMode` call in view 2). The views combine the two.

4.1 One frame

ImGui is an *immediate-mode* library: the whole UI is described again every frame by plain function calls. The loop in `main.cpp` runs in this order:

1. Poll input, compute `dt`, start a new ImGui frame.
2. Menu bar and the H / L shortcuts.
3. Dock space over the whole window. On the first frame, build the default layout if `imgui.ini` has none.
4. **Node Editor** and **Settings**. These run first, so edits made this frame are visible in the views this frame.
5. `graph.Animate(dt)` spins the Rotate nodes that have *spin* on, unless play mode is active. The divide slider is animated if requested.
6. Projection nodes receive the camera window's aspect ratio, measured last frame.
7. `graph.Evaluate()` produces the `EvalResult`: every object's `M`, plus `C`, `V`, `P` and warnings. The frustum mesh is rebuilt from it.
8. The three views render into their framebuffers and show them as images. In play mode view 3 also writes a new camera pose into the graph, which the next frame's evaluation picks up. Then the

Matrices panel.

9. `ImGui::Render`, draw the UI into the main window, draw any windows dragged outside the app, swap buffers.

Key idea: the 3D views do not draw directly to the screen. Each renders into its own offscreen texture during step 8, and `ImGui` draws that texture like a picture inside a window in step 9. That is what makes the views dockable, resizable and able to leave the main window.

5. The node graph

5.1 Data model

A `Node` holds an `id`, a `NodeType` and the parameters of every type: vector, angle and spin for transforms; a 4x4 `custom` matrix for Matrix nodes; shape and three triangle corners `tri[3]` for objects; position and target for the camera; and fov, near, far and aspect for projections. It is a flat struct rather than a class hierarchy, so duplicating a node is a single assignment. Evaluation fills three output fields:

`localMatrix`, `cumulativeMatrix` and `kind`, the chain the node belongs to.

A `Link` connects an output *attribute* (a pin) to an input attribute. `imnodes` needs a unique integer for every pin, so pin ids are derived from the node id:

Pin	Id	Used by
output	$id \cdot 8 + 1$	every node except MVP Output
input / Model	$id \cdot 8 + 2$	transforms; the Output's Model pin
View	$id \cdot 8 + 3$	MVP Output only
Projection	$id \cdot 8 + 4$	MVP Output only

`NodeOfAttr(attr) = attr / 8` recovers the node, and `attr % 8 == 1` identifies outputs. No lookup table is needed.

Link rules (AddLink)

- The two pins are normalised to *output* → *input*. Two outputs, two inputs, or a node linked to itself are rejected.
- Most inputs accept one link, so a new link replaces the old one. The Output's **Model** pin accepts many, one per object.
- An output can feed many inputs, so two objects can share a transform.

5.2 Evaluating chains (Resolve)

Evaluation walks *backwards* from each node to the source of its chain. `Resolve(n)` works as follows:

- Recompute `n`'s local matrix from its current parameters.
- Sources start a chain. **Object** gets kind *Model* and cumulative = identity. **Camera** gets kind *CameraPose* and cumulative = its base pose. **Projection** gets kind *Projection*.
- A transform resolves its upstream node first, then inherits its kind and sets `cumulative = local · upstream.cumulative`. This works after any source, including a Projection.
- A transform with nothing upstream starts a free-standing chain (kind *None*). A lone **Matrix** node is the typical case.

Results are memoised with a `state` array: 0 unvisited, 1 in progress, 2 done. Meeting a node that is *in progress* means the user built a cycle, and that node is marked Invalid instead of recursing forever. The editor colours nodes by `kind`, so an invalid chain turns red.

5.3 Producing the EvalResult

`Evaluate` resolves every node, then reads the three pins of the MVP Output:

- **Model:** every linked chain becomes an `ObjectInstance` holding its `M`. An Object chain copies its object's shape and triangle corners. Any other chain, such as a lone Matrix, is drawn as a cube.
- **View:** any chain gives the pose `C`, so a Matrix node can be the camera. A singular `C` has no inverse, so the default camera at (0, 2, 7) looking at (0, 0.5, 0) is used instead, as it is when nothing is connected. Then `V = inverse(C)`.
- **Projection:** any chain gives `P`. Without one, `P` = identity (section 3.5). Near, far and the visible volume are then read back out of `P` (section 3.6).
- Anything unexpected adds a human-readable warning, shown above the node canvas.

`Animate(dt)` adds `spinSpeed · dt` degrees to every Rotate node with *spin* on. `ResetGraph` replaces the graph with the starter example or a lone MVP Output. It keeps the id counters running, so imnodes never sees an old id reused for a new node.

5.4 Writing a camera pose back (SetCameraPose)

Play mode needs the opposite of evaluation: given the pose `C'` the user flew to, change the graph so that it produces `C'`. The View chain is `C = T · L`, where `L` is the matrix of its first node and `T` the product of everything after it. `T` stays as it is, so only `L` has to change:

$$L' = T^{-1} \cdot C' \text{ with } T = C \cdot L^{-1}, \text{ so } L' = L \cdot C^{-1} \cdot C'$$

The last form needs only `C` to be invertible, which evaluation already guarantees. How `L'` is stored depends on the first node:

- **Matrix node:** `custom = L'`, exactly.
- **Camera node:** `position` is `L'`'s translation column and the view direction is `-L'`'s third column. The look-at target is placed along that direction at the old target distance (at least 1). Roll is dropped, because a position plus a look-at point cannot express it.
- **Nothing connected:** a new Camera node is created, linked to the View pin and given the pose.
- **Anything else first** (for example a Translate) can't express a free pose, so play mode stops with a message.

`AddNode` appends to a `std::vector`, which can move every node in memory. The code therefore keeps the Output node's `id` across that call, never a pointer. A unit test caught exactly this bug during development.

6. Rendering infrastructure

6.1 Meshes

Every mesh uses the same vertex layout, `Vertex { px, py, pz, r, g, b }`: six floats, 24 bytes.

`Mesh::Upload` creates a vertex array object (VAO) and vertex buffer (VBO) once, describes attribute 0 (position, offset 0) and attribute 1 (colour, offset 12), and replaces the buffer contents on every call.

Builder	Draws as	Notes
<code>CubeVerts</code>	triangles	Unit cube centred at the origin, a different colour per face so orientation is readable.
<code>TriangleVerts(corners)</code>	triangles	A flat orange triangle from three corners. Each triangle object has its own corners, so the shared <code>triangle</code> mesh is re-uploaded (<code>GL_DYNAMIC_DRAW</code>) just before each triangle is drawn: three vertices, so it costs nothing.
<code>GridVerts(12)</code>	lines	Ground grid from -12 to 12. Each line is cut into 1-unit segments so it can <i>bend</i> when the divide warps it.
<code>AxesVerts</code>	lines	Red X, green Y, blue Z, 1.5 units long.
<code>NdcBoxVerts</code>	lines	The 12 edges of the cube $[-1, 1]^3$, plus two centre lines on its floor.
<code>FrustumVerts</code>	lines	The student camera's visible volume, in camera space, rebuilt every frame (below).

The frustum mesh

The visible volume is whatever `P` maps onto the NDC cube, so its eight corners are the cube's corners mapped back through the inverse:

$$\text{corner} = P^{-1} \cdot (\pm 1, \pm 1, \pm 1, 1), \text{ then divided by its own } w$$

For a perspective `P` this is the familiar pyramid: at distance d the rectangle has half-height $d \cdot \tan(\text{fov} / 2)$ and half-width `aspect` times that. For an orthographic matrix it is a box, for the identity the 2×2 box around the camera. Only a singular `P` falls back to the Projection node's fov, near and far.

The near and far rectangles plus four connecting edges make 12 lines, the first 24 vertices (`kFrustumWarpable`). They are drawn with the perspective-divide warp enabled, and their corners have NDC coordinates of exactly ± 1 , so at full divide they land on the edges of the NDC cube. The remaining lines are drawn unwarped: the eye-to-near-plane lines, only when `P` is perspective, and a green "up" tick.

6.2 Offscreen framebuffers

`ViewportFBO` owns a framebuffer with an RGB colour **texture** and a 24-bit depth **renderbuffer**. `Resize` reallocates both only when the window size changes. The texture is what ImGui later draws. OpenGL's texture origin is the bottom-left and ImGui's is the top-left, so `ShowViewport` passes UVs (0, 1) to (1, 0) to flip the image.

6.3 The Renderer

There is a single shader program. `Renderer` caches its uniform locations in `Init`, then each view follows the same pattern:

```
renderer.Begin(fbo, viewSetup);    // bind + clear the target, upload per-view uniforms
renderer.Draw(mesh, model, opts);  // per draw: model matrix, tint, warp/dim switches
...
renderer.End();                    // unbind
```

ViewSetup field	Meaning
<code>obsView, obsProj</code>	The camera <i>you</i> look through in this window: the orbit camera in views 1 and 2, the student's camera in view 3.
<code>toEye</code>	Takes this view's scene space to the student's camera space. <code>V</code> when the scene is the world, identity when the scene already is camera space.
<code>camProj</code>	The student's <code>P</code> , used for the warp and for the inside/outside test.
<code>warp, depthMode, boxHalf, boxCenter</code>	Perspective-divide visualisation in view 2 (section 7).
<code>dimOutside, clipOutside</code>	Fade, or remove, what the student's camera cannot see.
<code>nearPlane, farPlane, bg</code>	Used by the depth mode, the clip-behind test and the fade colour.

DrawOpts field	Meaning
<code>tint, tintAmount</code>	Blend the mesh colour towards a flat colour: grey camera body, blue floor, fading NDC box.
<code>warp</code>	Whether this draw follows the divide. Off for helpers such as the camera body and the floor.
<code>dim</code>	Whether this draw is dimmed or clipped by the student's frustum. Off for axes, camera and frustum.

6.4 Scene helpers

`Scene` owns the shared meshes. `DrawWorld(renderer, scene, ev, worldToScene)` draws the grid, the axes and every object with model matrix `worldToScene * M`. Passing identity draws the world as-is (views 1 and 3); passing `V` draws it moved into camera space (view 2). That one parameter is the whole difference between "what you think" and "what actually happens". Two flags let view 2 skip the world grid and view 3 hide grid and axes. `DrawCameraGizmo` draws the camera body, its yellow lens and the frustum at `camToScene: C` in view 1, identity in view 2.

6.5 The orbit camera

Views 1 and 2 are seen through an `OrbitCamera` that circles a target point using spherical coordinates (yaw ψ , pitch φ , distance r):

$$\text{eye} = \text{target} + r \cdot (\cos \varphi \cdot \sin \psi, \sin \varphi, \cos \varphi \cdot \cos \psi), \text{view} = \text{lookAt}(\text{eye}, \text{target}, \text{up})$$

Dragging changes ψ and φ by 0.4° per pixel, with φ clamped to $\pm 89^\circ$ so `lookAt` never looks straight along up . The mouse wheel multiplies r by 0.9^{wheel} , so zoom feels the same at any distance.

7. The shaders, line by line

All three views use one vertex shader and one fragment shader, embedded as strings in `Renderer.cpp`. Their job is more than ordinary rendering: they can also *animate* the perspective divide and show which parts of the world the student's camera can see.

7.1 Vertex shader

Renderer.cpp: kVertSrc (comments shortened)

```
#version 330 core
layout(location = 0) in vec3 aPos;
layout(location = 1) in vec3 aColor;

uniform mat4 uModel;      // local -> scene space (the space the observer looks at)
uniform mat4 uToEye;      // scene space -> the student's camera eye space
uniform mat4 uCamProj;    // the student's projection matrix
uniform mat4 uObsView;    // observer camera of this viewport
uniform mat4 uObsProj;

uniform float uWarp;      // 0 = plain eye space, 1 = after dividing by w
uniform int  uDepthMode;  // 0 = distance remapped linearly to [-1,1], 1 = real NDC z
uniform float uNear;
uniform float uFar;
uniform float uBoxHalf;  // the NDC cube [-1,1]^3 is drawn with this half size...
uniform float uBoxCenter; // ...centered this far in front of the camera

out vec3 vColor;
out vec4 vCamClip;
out float vEyeDist;

void main() {
    vec4 scene = uModel * vec4(aPos, 1.0);
    vec4 eye = uToEye * scene;
    vec4 clip = uCamProj * eye;

    vec3 pos = scene.xyz;
    if (uWarp > 0.0) {
        float w = max(clip.w, 1e-3);
        vec3 ndc = clip.xyz / w;
        float zn = (uDepthMode == 0) ? 2.0 * (-eye.z - uNear) / (uFar - uNear) - 1.0 : ndc.z;
        vec3 projected = vec3(ndc.x, ndc.y, 0.0) * uBoxHalf
            + vec3(0.0, 0.0, -(uBoxCenter + zn * uBoxHalf));
        projected = clamp(projected, vec3(-300.0), vec3(300.0));
        pos = mix(scene.xyz, projected, uWarp);
    }

    gl_Position = uObsProj * uObsView * vec4(pos, 1.0);
    vColor = aColor;
    vCamClip = clip;
    vEyeDist = -eye.z;
}
```

Two pipelines in one shader. Each vertex is pushed through two cameras at once:

- **The student's camera:** `scene` → `eye` → `clip` using `uToEye` and `uCamProj`. This result is never drawn directly. It is used to decide what that camera can see, and to compute the warp.
- **The observer camera:** `uObsProj` · `uObsView` turns the (possibly warped) scene position into `gl_Position`, which the GPU rasterises.

In view 3 the observer is the student camera (`uObsView = V`, `uObsProj = P`), so `gl_Position = P · V · M · v`, the textbook pipeline. In views 1 and 2 the observer is the orbit camera, so you see the student's camera from outside.

The warp: animating the divide

Only view 2 sets `uWarp > 0`, and there the scene space *is* the camera space. The shader computes the real NDC position `ndc = clip.xyz / w`, then places the NDC cube back in front of the fixed camera so you can see it in 3D:

$$\text{projected} = (h \cdot x_{ndc}, h \cdot y_{ndc}, -(c + h \cdot z_n))$$

with `h = uBoxHalf = 4` and `c = uBoxCenter = near + h`. NDC `z = -1` lands at distance `c - h = near`, on the real near plane, and `z = +1` at `near + 2h`. Because `x_ndc = (g/a) · x / d`, the drawn `x` is `h · (g/a) · x / d`. That `1 / d` is what visibly shrinks the far objects as the slider moves.

`mix(scene, projected, uWarp)` interpolates each vertex linearly between where it was and where the divide puts it, so the slider animates smoothly. Straight lines bend while warping because each vertex moves by a different amount, which is why the grid is built from short segments.

Depth mode	z_n	Why
0: distance, spread evenly	$2 (d - n) / (f - n) - 1$	Linear in distance. Easy to read: equally spaced objects stay equally spaced.
1: real NDC z	<code>ndc.z</code>	What the depth buffer really stores. Everything crowds towards the far face (table in 3.3).

Two safety measures: `w` is clamped to at least 0.001, so points behind the camera do not divide by zero. The result is clamped to ± 300 , so those points do not shoot off to infinity. Such points are hidden anyway by the fragment shader's clip-behind test.

Outputs

`vCamClip` passes the *student's* clip coordinates to the fragment shader, and `vEyeDist` the distance in front of the student's camera.

7.2 Fragment shader

Renderer.cpp: `kFragSrc`

```
uniform vec3 uTint;
uniform float uTintAmount;
uniform int uDimOutside; // fade what the student's camera cannot see
uniform int uClipBehind; // drop fragments in front of the near plane
uniform int uClipOutside; // drop everything outside the [-1,1] cube
uniform float uNear;
uniform vec3 uBg;

void main() {
    if (uClipBehind == 1 && vEyeDist < uNear) discard;
    vec3 c = mix(vColor, uTint, uTintAmount);
    bool inside = vCamClip.w > 0.0 &&
        all(lessThanEqual(abs(vCamClip.xyz), vec3(vCamClip.w)));
    if (uClipOutside == 1 && !inside) discard;
    if (uDimOutside == 1 && !inside) c = mix(c, uBg, 0.72);
    FragColor = vec4(c, 1.0);
}
```

- **Clip behind:** while warping, anything closer than the near plane would project to nonsense, so it is discarded.
- **Tint:** `mix(colour, tint, amount)` is how the camera body turns grey, the floor blue, and the NDC box fades in. Its tint is the background colour, with amount `1 - warp`.

- **Inside test:** exactly the GPU clipping rule from section 3.4, $|x|, |y|, |z| \leq w$ with $w > 0$, evaluated per pixel on the interpolated clip coordinates. The clip coordinates are a linear function of position, so the interpolated value is the correct value at that surface point.
- **Dim or cut:** outside points are blended 72% towards the background, or removed when *cut away outside* [-1, 1] is on. Removal is what the real GPU does.

8. The three views

Each view is a struct in `Views.cpp` with a `Draw(ctx, open)` method. `FrameContext` bundles what all of them need: the renderer, the scene, this frame's `EvalResult`, the options, the graph and `dt` (the last two for play mode). All three follow the same steps: begin the ImGui window, size the framebuffer to the free space, fill a `ViewSetup`, draw, show the texture, then draw the overlays.

	1. Think	2. Actual	3. Camera
Scene space	world	student camera space	world
Objects drawn with	<code>M</code>	<code>V · M</code>	<code>M</code>
<code>toEye</code>	<code>V</code>	identity	<code>V</code>
Observer (<code>obsView</code> , <code>obsProj</code>)	orbit camera	orbit camera	<code>V, P</code> (the real camera)
Student camera drawn at	<code>C</code>	identity (origin)	not drawn
Floor grid	world grid	fixed floor under the camera	world grid, toggle
Warp	never	slider	never

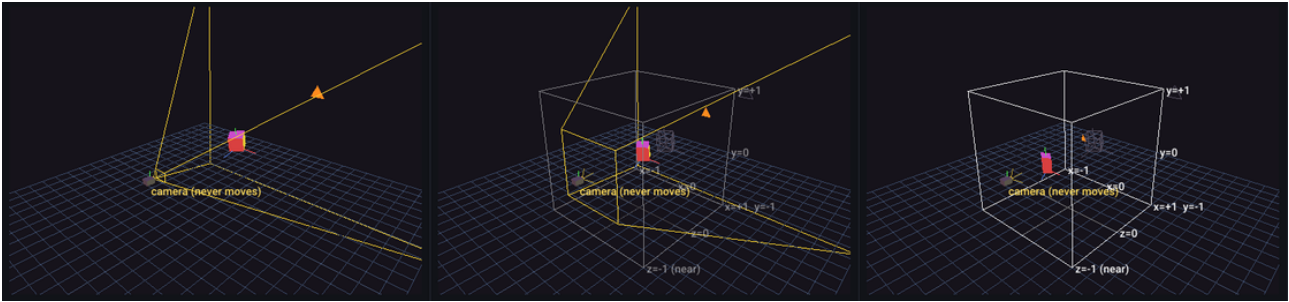
8.1 What you think happens

The world is drawn untouched (`worldToScene` = identity) and the camera gizmo is drawn at its pose `C`. Because `toEye` = `V`, the shader can still tell which parts of the world the student camera sees, and dims the rest. The overlay prints `C`'s fourth column, the camera position.

8.2 What actually happens

Here the scene is camera space. The world is drawn with `worldToScene` = `V`, so every object gets `V · M`, and the camera gizmo is drawn at the identity: at the origin, looking down -z, never moving. The world's own grid is not drawn. Instead a blue floor is drawn at a fixed position under the camera, so the only thing that visibly moves is the world.

The NDC cube is drawn with `cubeToEye` = `T(0, 0, -c) · S(h, h, -h)`, which maps cube coordinates to the same place the shader sends NDC points. The -h in z makes cube z = -1 the face nearest the camera. It fades in with the slider. With *ghost* on, the world is drawn a second time unwarped in wireframe (`glPolygonMode(GL_LINE)`) so you can compare before and after.



View 2 with the divide at 0, 0.5 and 1. The frustum (yellow) turns into the white NDC cube, and the triangle at the back shrinks far more than the cube at the front.

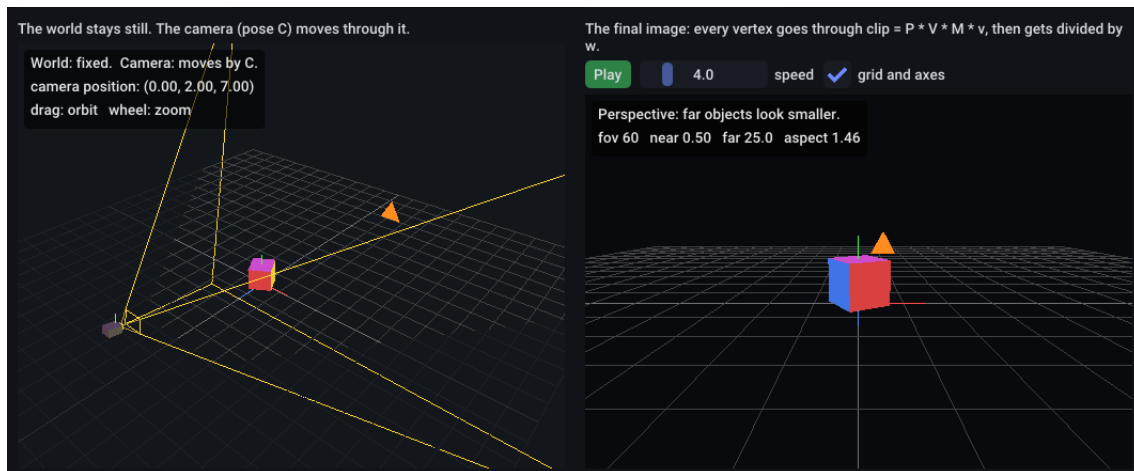
With no projection there is nothing to divide, so the slider is disabled, `cubeToEye` is the identity, and the cube shown is the real $2 \times 2 \times 2$ box around the camera. Text labels at 3D points use `Label3D`, which projects a point with the observer's matrices and converts NDC to pixels:

$$pixel_x = left + (x_{ndc} \cdot 0.5 + 0.5) \cdot width, pixel_y = top + (0.5 - y_{ndc} \cdot 0.5) \cdot height$$

The y term is flipped because NDC y points up while screen y points down. Points behind the observer ($w \leq 0$) or off-screen are skipped.

8.3 What the camera sees

The observer is the student's camera itself: `obsView = V`, `obsProj = P`. Nothing is warped or dimmed. The GPU does the real divide, clipping and depth test. This window also measures its own aspect ratio, which `main.cpp` feeds back into every Projection node before the next evaluation, so the image is never stretched (one frame of lag, invisible in practice).



View 1 (left) shows the camera and its frustum in the world. View 3 (right) is the image that camera produces.

8.4 Play mode inside

`CameraView::Fly` runs every frame while playing. It starts from the current pose `C` and turns it into a position and a direction:

$$pos = C[3], fwd = -normalize(C[2]), yaw = atan2(fwd_x, -fwd_z), pitch = asin(fwd_y)$$

- **Look:** while the image is dragged, yaw and pitch change by 0.004 radians per pixel. Pitch is clamped to $\pm 89^\circ$ so `lookAt` never looks straight up or down. The direction is rebuilt as `fwd = (cos p · sin y, sin p, -cos p · cos y)`.
- **Move:** `right = normalize(fwd × up)`. W/S add $\pm fwd$, A/D $\pm right$, Q/E $\pm world\ up$. The sum is normalised, so diagonal moves aren't faster, then scaled by `speed · dt`, times 3 with Shift.
- **Write back:** the new pose `inverse(lookAt(pos, pos + fwd, up))` goes to `NodeGraph::SetCameraPose` (section 5.4). The next frame's evaluation turns the edited nodes back into `C`, so every view and the Matrices panel follow.

Keys only act while the camera window has focus, and the Play button gives it focus. Esc stops play mode. `main.cpp` skips `Animate` while playing, otherwise a spinning node after the Camera would keep turning the view.

9. The user interface

9.1 Node editor

The editor is built with **imnodes**, which, like `ImGui`, is immediate mode. Every frame `DrawNodeEditor` calls `BeginNodeEditor`, emits each node (title bar, input pins, widgets, matrix readout, output pin) and each link, then `EndNodeEditor`. Only after `EndNodeEditor` can it ask what the user did: `IsLinkCreated`, `IsLinkDestroyed`, hovered node or link, selection.

- **Adding:** the toolbar buttons and the right-click menu call `AddNode` and place the node with `SetNodeScreenSpacePos`. Toolbar nodes appear in the middle of the canvas, offset in a small cascade so they don't stack exactly. Names get a counter ("Translate 2").
- **Deleting:** the x button, the Delete or Backspace key, and the menus only *queue* ids. Deletion happens after `EndNodeEditor`, so `imnodes` never holds stale references during the frame. Removing a node also removes its links.
- **Duplicating:** copies the whole `Node` struct, then gives it a new id and name. Links are not copied.
- **Colours:** title bars are coloured by type for sources and by chain `kind` for transforms.
- **Input contrast:** field backgrounds are pushed a little lighter inside the editor so they stand out on the node bodies.

The Matrix and triangle editors



Left: a Matrix node holding an orthographic projection. Right: an Object node set to Triangle, with its three corners.

GLM stores matrices column by column (`m[col][row]`), but people write them row by row. `DrawMatrixEditor` therefore shows four `DragFloat4` rows: row r is copied out as `(m[0][r], m[1][r], m[2][r], m[3][r])`, edited, and written back to the same places. *Identity*, *Transpose* and *Invert* call the GLM functions. *Invert* is disabled when $|\det| < 10^{-6}$, and the determinant is printed under the buttons.

When an Object's shape is Triangle, three `DragFloat3` fields edit `tri[0..2]` directly. *Reset triangle* copies the corners from a default-constructed `Node`.

9.2 Docking and windows

The app uses the *docking* branch of Dear ImGui (v1.90.9-docking). `DockSpaceOverViewport` turns the whole main window into a dock area. On the first frame, if no saved layout exists, `BuildDefaultLayout` uses the DockBuilder API to split it: 48% bottom for the editor, the top in thirds for the views, and 24% of the bottom on the right for Settings and Matrices as tabs. ImGui saves whatever the user arranges to `imgui.ini`.

Multi-viewports (`ImGuiConfigFlags_ViewportsEnable`) let a window be dragged out of the app to become a real operating-system window. ImGui's GLFW backend creates each such window with an OpenGL context that *shares* objects with the main one, so the views' textures can be shown there with no extra work. Our own drawing happens while the main context is current. After drawing the extra windows, `main.cpp` makes the main context current again. This ImGui version cannot position windows under Wayland, so on Linux GLFW is asked for X11 (XWayland on a Wayland desktop).

`ConfigWindowsMoveFromTitleBarOnly` is on, so dragging inside a 3D view orbits it (or looks around in play mode) instead of moving the window. Each view's image is covered by an `InvisibleButton` that reports dragging and hovering.

9.3 Options, theme and text

`Options` (UI.h) is a plain struct of toggles shared by Settings, the node editor and the views: animation, dimming, matrix readouts, the divide controls, the camera view's grid, and the two text switches.

`Explanations` (H) hides the descriptions and overlay boxes. `Labels` (L) hides the text drawn at 3D points.

`ApplyDarkTheme` sets ImGui's spacing, rounding and every colour from a small palette with one accent blue (0x6C8CFF), and themes imnodes to match. `LoadFonts` loads Roboto for the UI and Cousine, a monospace font, for matrices, both shipped with Dear ImGui and copied next to the executable by CMake. Sizes are multiplied by the monitor's content scale for high-DPI screens.

10. Extending the app

To add...	Change
A transform node	Add it to <code>NodeType</code> and <code>IsTransform</code> , give it a matrix in <code>Node::RecomputeLocal</code> , add a toolbar entry in <code>kTransformKinds</code> and its widgets in the <code>switch</code> inside <code>DrawNodeEditor</code> (UI.cpp).
An object shape	Write a builder next to <code>CubeVerts</code> (Renderer.cpp), add a <code>Mesh</code> to <code>Scene</code> and upload it in <code>Scene::Init</code> , add a value to <code>ObjectShape</code> , pick the mesh in <code>DrawWorld</code> , and add the name to the shape combo in <code>DrawNodeEditor</code> . Per-object vertex data, like the triangle's corners, goes in <code>Node</code> and <code>ObjectInstance</code> .
A projection type	Usually no code: type the matrix into a Matrix node. For a dedicated node, add parameters to <code>Node</code> and build the matrix in <code>RecomputeLocal</code> . The views derive the volume from P on their own.
A window	Title and visibility flag in UI.h, a slot in <code>BuildDefaultLayout</code> , a View-menu entry and a draw call in <code>main.cpp</code> .
A shader effect	New uniform in the shader strings, its location in <code>Renderer::Init</code> , a field in <code>ViewSetup</code> (per view) or <code>DrawOpts</code> (per draw), and upload it in <code>Begin</code> or <code>Draw</code> .
A setting	A field in <code>Options</code> , a widget in <code>DrawSettingsWindow</code> , and read it wherever it applies.
This guide	Edit <code>docs/guide/make_guide.py</code> and run it. Screenshots live in <code>docs/guide/fig/</code> .

Glossary

Term	Meaning
Homogeneous coordinates	(x, y, z, w). Points use w = 1 so translation fits in a 4×4 matrix. After projection w holds the depth to divide by.
Pose C	The camera's placement in the world, camera → world.
View matrix V	C^{-1} : world → camera. Moves the world so the camera sits at the origin looking down -z.
Clip space	Output of P, before the divide. Clipping happens here: $ x , y , z \leq w$.
NDC	Normalised device coordinates: clip / w. The visible volume is the cube $[-1, 1]^3$.
Frustum	The truncated pyramid a perspective camera sees, between the near and far planes.
Determinant	A single number from a matrix. Zero means the matrix squashes space flat and has no inverse.
FBO	Framebuffer object: an offscreen render target. Each view renders into one.
Immediate mode	A UI style where the whole interface is re-declared every frame by function calls (ImGui, imnodes).

